

Operating Systems [CS F372]

Semester II, 2025 – 2026



BITS Pilani
K K Birla Goa Campus

Kernel Level Custom Packet Network Filter (Netfilter)

Project Report

Course Instructor: Dr Biju K Raveendran

Acknowledgements

We would like to thank Prof. Biju Raveendran of the CSIS Department at BITS Goa for giving us the opportunity to create this project as part of the Operating Systems course in the second semester of the academic year 2025-2026.

Link to a video demonstrating this application is given below:

 VIDEO-2026-04-28-21-47-56.mp4

TABLE OF CONTENTS

1. Project Overview	4
2. Environment Setup & Compilation	5
2.1 Prerequisites and Toolchain	5
2.2 Local Build Configuration (Makefile)	6
3. Kernel Module Implementation	7
3.1 Kernel Space Architecture	7
3.2 Dynamic Configuration Interface (/proc)	8
3.3 Core Filtering Logic (firewall.c)	8
4. Testing and Validation	9
4.1 Module Compilation and Loading	9
4.2 Policy Enforcement via Wrapper Script	9
Test 1: Domain Blocking via IP Resolution	10
Test 2: Deep Packet Inspection for URLs	10
Test 3: Destination IP Blocking	11
Test 4: Destination Port Blocking	11
Test 5 and 6: Source IP and Source Port Blocking	12
Test 7: Session Termination and Logging	12
5. Conclusion	13
5.1 Conclusion	13
5.2 Limitations	13

1. Project Overview

The custom Linux kernel firewall is a high-performance Loadable Kernel Module (LKM) engineered to provide near-zero latency network traffic management. By operating directly within the kernel space, this module bypasses the overhead of standard user-space applications, intercepting and evaluating every incoming and outgoing packet at the protocol level.

Core System Architecture The firewall leverages the Linux Netfilter subsystem to establish strict control over the network stack. The operational lifecycle is divided into three distinct phases:

- **Registration Phase:** Upon initialization using the `insmod` command, the module seamlessly registers custom hook functions directly into the kernel's networking stack, establishing the first line of defense.
- **Monitoring Phase:** As packets traverse the network interface, the Netfilter hooks intercept them, parsing the `sk_buff` data structures to extract critical protocol headers, including source/destination IP addresses and transport-layer ports.
- **Enforcement Phase:** The module cross-references the extracted packet data against dynamically loaded restriction rules. If a violation is detected, the kernel is instructed to immediately drop (`NF_DROP`) the packet; otherwise, the packet is accepted (`NF_ACCEPT`) and allowed to continue its routing path.

Key Features and Capabilities Moving beyond standard static filtering, this project successfully implements several advanced networking features:

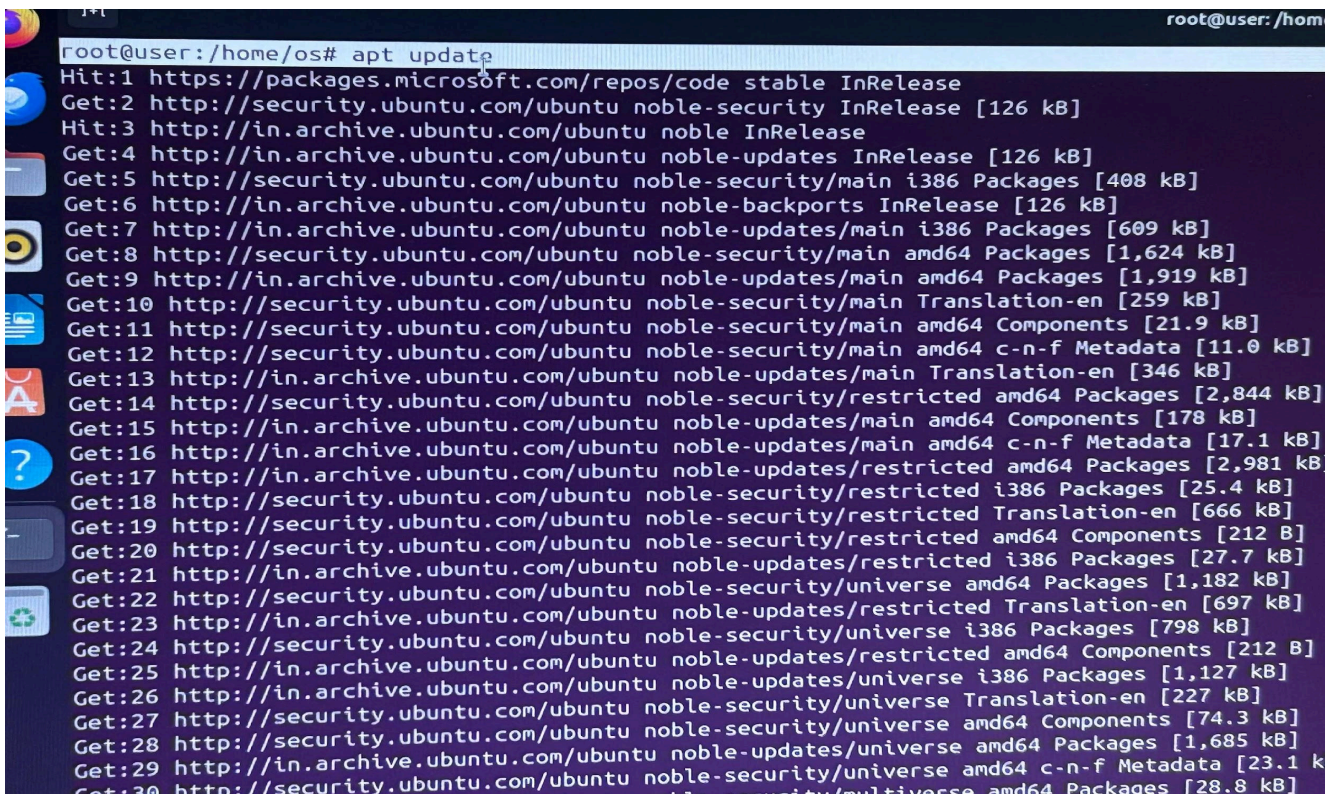
- **Dynamic Runtime Configuration:** Through a dedicated interface, filtering rules can be injected or removed on the fly without requiring the kernel module to be unloaded or the system to be rebooted.
- **Protocol-Level Granularity:** The firewall supports absolute control over specific transport layer attributes, capable of blocking exact destination ports (e.g., Port 443 for HTTPS) or specific IPv4 addresses.
- **Deep Packet Inspection (DPI):** The module features an application-layer inspection mechanism capable of scanning unencrypted UDP payloads (specifically targeting DNS traffic on Port 53) to identify and drop requests resolving to restricted URL strings.
- **Immutable Audit Logging:** Every intercepted and dropped packet triggers a kernel-level logging event (`printk`), generating a persistent and highly detailed audit trail within the kernel ring buffer for post-session analysis.

2. Environment Setup & Compilation

2.1 Prerequisites and Toolchain

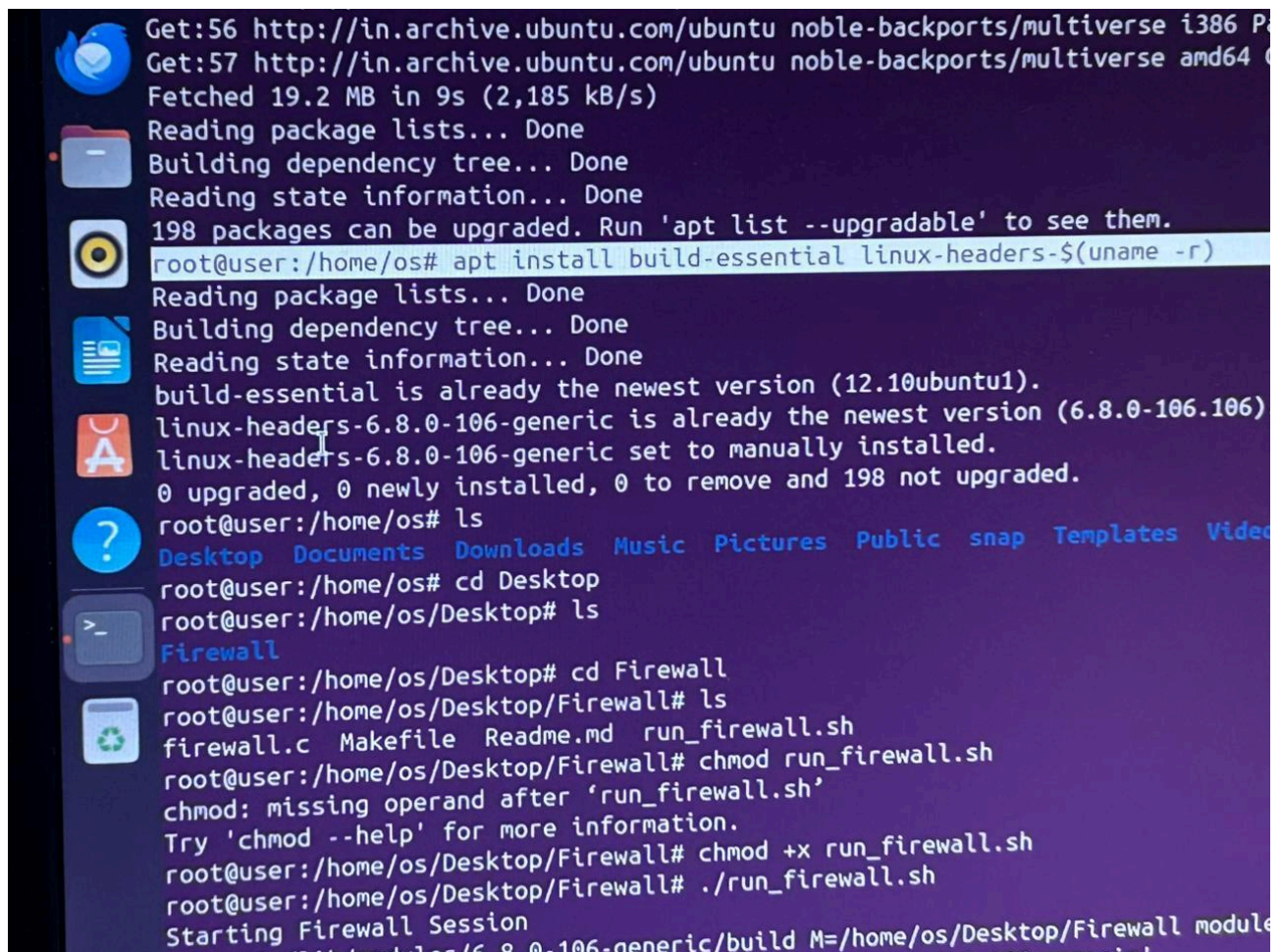
To ensure the custom firewall module interacts correctly with the host machine's networking stack, a specific development toolchain must be established. Because the project modifies kernel-space behavior, root privileges are strictly mandated to load (insmod) and unload (rmmod) the kernel modules.

The system repositories were first updated to fetch the latest package lists. Following this, the build-essential package—which provides the necessary gcc compiler and make utility—was verified. Crucially, the Linux headers matching the exact current kernel version (linux-headers-6.8.0-106-generic) were installed. Compiling the module against the precise headers of the actively running kernel is a strict requirement to ensure structural compatibility with the kernel's internal API and prevent system instability. Finally, standard executable permissions were granted to the wrapper script using `chmod +x run_firewall.sh` to facilitate the automated user-space operations.



```
root@user:/home/os# apt update
Hit:1 https://packages.microsoft.com/repos/code stable InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:3 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:4 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main i386 Packages [408 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu noble-updates/main i386 Packages [609 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,624 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,919 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [259 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.9 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [11.0 kB]
Get:13 http://in.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [346 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2,844 kB]
Get:15 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [178 kB]
Get:16 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [17.1 kB]
Get:17 http://in.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2,981 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/restricted i386 Packages [25.4 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [666 kB]
Get:20 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:21 http://in.archive.ubuntu.com/ubuntu noble-updates/restricted i386 Packages [27.7 kB]
Get:22 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,182 kB]
Get:23 http://in.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [697 kB]
Get:24 http://security.ubuntu.com/ubuntu noble-security/universe i386 Packages [798 kB]
Get:25 http://in.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:26 http://in.archive.ubuntu.com/ubuntu noble-updates/universe i386 Packages [1,127 kB]
Get:27 http://in.archive.ubuntu.com/ubuntu noble-security/universe Translation-en [227 kB]
Get:28 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.3 kB]
Get:29 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,685 kB]
Get:30 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [23.1 kB]
Get:31 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [28.8 kB]
```

Figure 1: Updating system repositories via apt update



```
Get:56 http://in.archive.ubuntu.com/ubuntu noble-backports/multiverse i386 P
Get:57 http://in.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64
Fetched 19.2 MB in 9s (2,185 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
198 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@user:/home/os# apt install build-essential linux-headers-$(uname -r)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.10ubuntu1).
linux-headers-6.8.0-106-generic is already the newest version (6.8.0-106.106)
linux-headers-6.8.0-106-generic set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 198 not upgraded.
root@user:/home/os# ls
Desktop Documents Downloads Music Pictures Public snap Templates Videos
root@user:/home/os# cd Desktop
root@user:/home/os/Desktop# ls
Firewall
root@user:/home/os/Desktop# cd Firewall
root@user:/home/os/Desktop/Firewall# ls
firewall.c Makefile Readme.md run_firewall.sh
root@user:/home/os/Desktop/Firewall# chmod run_firewall.sh
chmod: missing operand after 'run_firewall.sh'
Try 'chmod --help' for more information.
root@user:/home/os/Desktop/Firewall# chmod +x run_firewall.sh
root@user:/home/os/Desktop/Firewall# ./run_firewall.sh
Starting Firewall Session
```

Figure 2: Verifying installation of Linux headers and build dependencies, and setting executable permissions for bash script

2.2 Local Build Configuration (Makefile)

A local Makefile was created to instruct the kernel build system to compile the firewall.c source code against the active system's kernel tree.

The directive `obj-m += firewall.o` is utilized to specify that the compiled object should be built as a dynamically loadable kernel module (LKM), rather than being statically linked into the core kernel image.

The default target executes the command `make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules`. This command performs two critical actions:

1. The `-C` flag changes the directory to the running kernel's build directory, effectively invoking the kernel's own highly complex Makefile.
2. The `M=$(PWD)` argument instructs the kernel's build system to return to the project's current working directory to locate the external module source code and build it there.

Additionally, a clean target is implemented (`make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean`) to safely and efficiently remove compiled binaries, object files (.o), and module symbol versions (.mod.c, .ko) from previous builds.

3. Kernel Module Implementation

3.1 Kernel Space Architecture

The core architecture leverages the Linux Netfilter subsystem, a framework inside the Linux kernel that allows custom handler functions to intercept and manipulate network packets. The firewall module initializes by defining `nf_hook_ops` structures and binding the `block_handler` function to two critical hook points:

- `NF_INET_PRE_ROUTING`: Intercepts and inspects all incoming packets immediately upon arrival from the network interface, before any routing decisions are made by the kernel.
- `NF_INET_POST_ROUTING`: Intercepts all outgoing packets just before they are placed on the network wire for transmission.

For both hooks, the protocol family is strictly defined as `NFPROTO_IPV4` to filter standard IPv4 traffic. Furthermore, the hooks are assigned the highest possible execution priority (`NF_IP_PRI_FIRST`). This ensures that the custom firewall acts as the absolute first line of defense, processing packets before any other kernel.

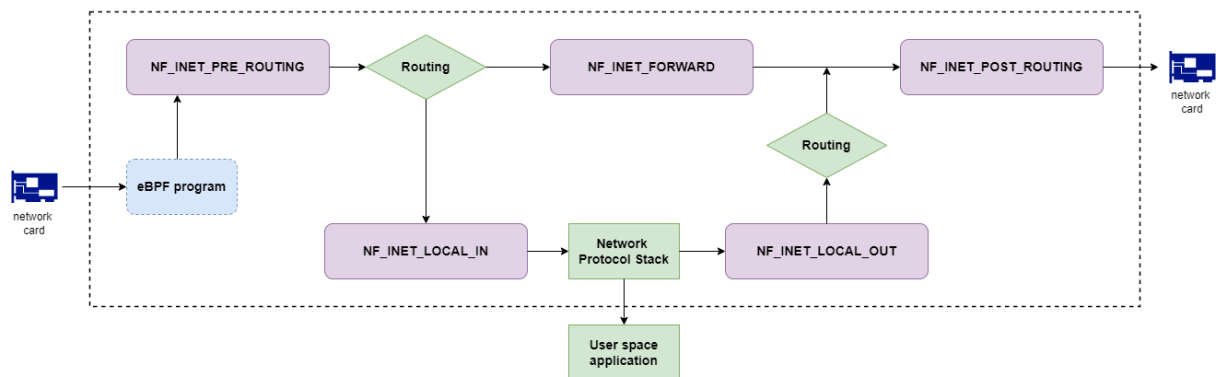


Figure 3: Linux Netfilter Packet Traversal Flow

3.2 Dynamic Configuration Interface (/proc)

To achieve dynamic rule updates without requiring a module reload (which would interrupt network flow), a dedicated virtual filesystem interface was implemented. During module initialization, `proc_mkdir` establishes a directory at `/proc/my_firewall`.

This `/proc` file acts as the secure bridge between user-space inputs and kernel-space data structures. The `rule_write` function processes incoming commands through the following mechanisms:

1. **Memory Isolation:** It utilizes `copy_from_user()` to safely transfer the input string from user-space memory into a kernel-space buffer.
2. **Command Parsing:** The buffer is parsed using `sscanf` to extract the directive (e.g., `source_ip`, `block_url`) and the corresponding value.
3. **Data Conversion:** String-based IP addresses are converted into 32-bit big-endian network byte order using `in_aton()`, while port strings are safely converted to 16-bit unsigned integers using `kstrtoul6()`.

Because network traffic (which triggers the hook functions) and user-space configuration commands operate concurrently, a spinlock (`DEFINE_SPINLOCK(fw_lock)`) is utilized. Specifically, the `spin_lock_bh()` variant is used to acquire the lock and simultaneously disable software interrupts (bottom halves) on the local CPU, preventing deadlocks or race conditions when writing to the configuration arrays (`blocked_source_ips`, `blocked_urls`, etc.) while a packet is being actively processed.

3.3 Core Filtering Logic (firewall.c)

The main filtering engine, `block_handler`, takes the `sk_buff` (socket buffer)—the fundamental data structure representing a network packet in the Linux kernel—and evaluates it against the active ruleset. By default, the handler initializes the action as `ACCEPT` (`NF_ACCEPT`), only changing to `DROP` (`NF_DROP`) if a violation is detected.

- **IP & Port Blocking:** The handler uses the `ip_hdr(skb)` helper to extract the IP header and evaluate the source (`saddr`) and destination (`daddr`) IP addresses. To check ports, it verifies if the protocol is `IPPROTO_TCP` or `IPPROTO_UDP`. If true, it extracts the respective transport header (`tcphdr` or

udphdr) and converts the port numbers from network byte order to host byte order using ntohs() for comparison against the blocked port lists.

- **URL Packet Inspection (DPI):** For domain blocking, the firewall utilizes a lightweight Deep Packet Inspection (DPI) approach. The `is_website_blocked` function isolates UDP traffic where either the source or destination port is 53 (standard DNS traffic). It calculates the start of the payload data by advancing a pointer past the UDP header size (`sizeof(struct udphdr)`) and calculates the payload length. Using the `strnstr()` function, it searches the raw packet payload for strings matching the registered restricted URLs. If a match is found, the DNS resolution request is immediately dropped.
- **Audit Logging:** Whenever a packet is dropped due to a rule match, `printk(KERN_INFO...)` is invoked to record the specific violation (e.g., "Dropped: src=...", "Dropped: website") directly into the kernel ring buffer, providing an immutable audit trail of the firewall's interventions.

4. Testing and Validation

4.1 Module Compilation and Loading

A Bash wrapper script (`run_firewall.sh`) manages the operational lifecycle of the firewall. Executing the script first checks for root privileges, as they are mandatory for modifying kernel states. To ensure a clean audit trail, it clears the existing kernel ring buffer using the `dmesg -c` command, preventing logs from previous sessions from contaminating the current run. It then triggers the `make` command, which compiles the `firewall.ko` module using the system's `gcc-13` compiler, and subsequently inserts it into the active kernel using `insmod`.

4.2 Policy Enforcement via Wrapper Script

During execution, the script presents an interactive menu via a continuous while loop to dynamically assign filtering rules. The following tests were executed to validate the firewall's enforcement capabilities:

Test 1: Domain Blocking via IP Resolution

Using Option 1, the script successfully resolved the domain neverssl.com to the IPv4 address 34.223.124.45. This resolution happens strictly in user space using the `dig +short` command to minimize kernel overhead. The script then echoes this resolved IP into the kernel module using the format `destination_ip 34.223.124.45` directed at the `/proc/my_firewall/rules` interface.

```
root@user:/home/...:~# chmod: missing operand after 'run_firewall.sh'
Try 'chmod --help' for more information.
root@user:/home/os/Desktop/Firewall# chmod +x run_firewall.sh
root@user:/home/os/Desktop/Firewall# ./run_firewall.sh
Starting Firewall Session
make -C /lib/modules/6.8.0-106-generic/build M=/home/os/Desktop/Firewall modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-106-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
CC [M] /home/os/Desktop/Firewall/firewall.o
MODPOST /home/os/Desktop/Firewall/Module.symvers
CC [M] /home/os/Desktop/Firewall/firewall.mod.o
LD [M] /home/os/Desktop/Firewall/firewall.ko
BTF [M] /home/os/Desktop/Firewall/firewall.ko
Skipping BTF generation for /home/os/Desktop/Firewall/firewall.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-106-generic'
Firewall module loaded.

Choose an option:
1) Block a Website Via IP Resolution Ex: neverssl.com. Works on websites with only a single IP Address
2) Block a Website via Packet Inspection. Ex: cnn (will block websites on basis of its name)
3) Block a Destination IP
4) Block a Destination Port
5) Block a Source IP
6) Block a Source Port
7) End Session & Save Logs
Selection (In between 1-7): 1
Enter website (e.g. example.com): [Do not enter https ]neverssl.com
Resolved to 34.223.124.45.
```

Figure 4: Compilation, module loading, and successful user-space IP resolution for neverssl.com.

Test 2: Deep Packet Inspection for URLs

Using Option 2, the raw string `cnn` was registered into the kernel's URL blocking array. The script passes the command `block_url cnn` to the `procfs` file, instructing the kernel to intercept and drop any DNS queries containing that string within the payload.

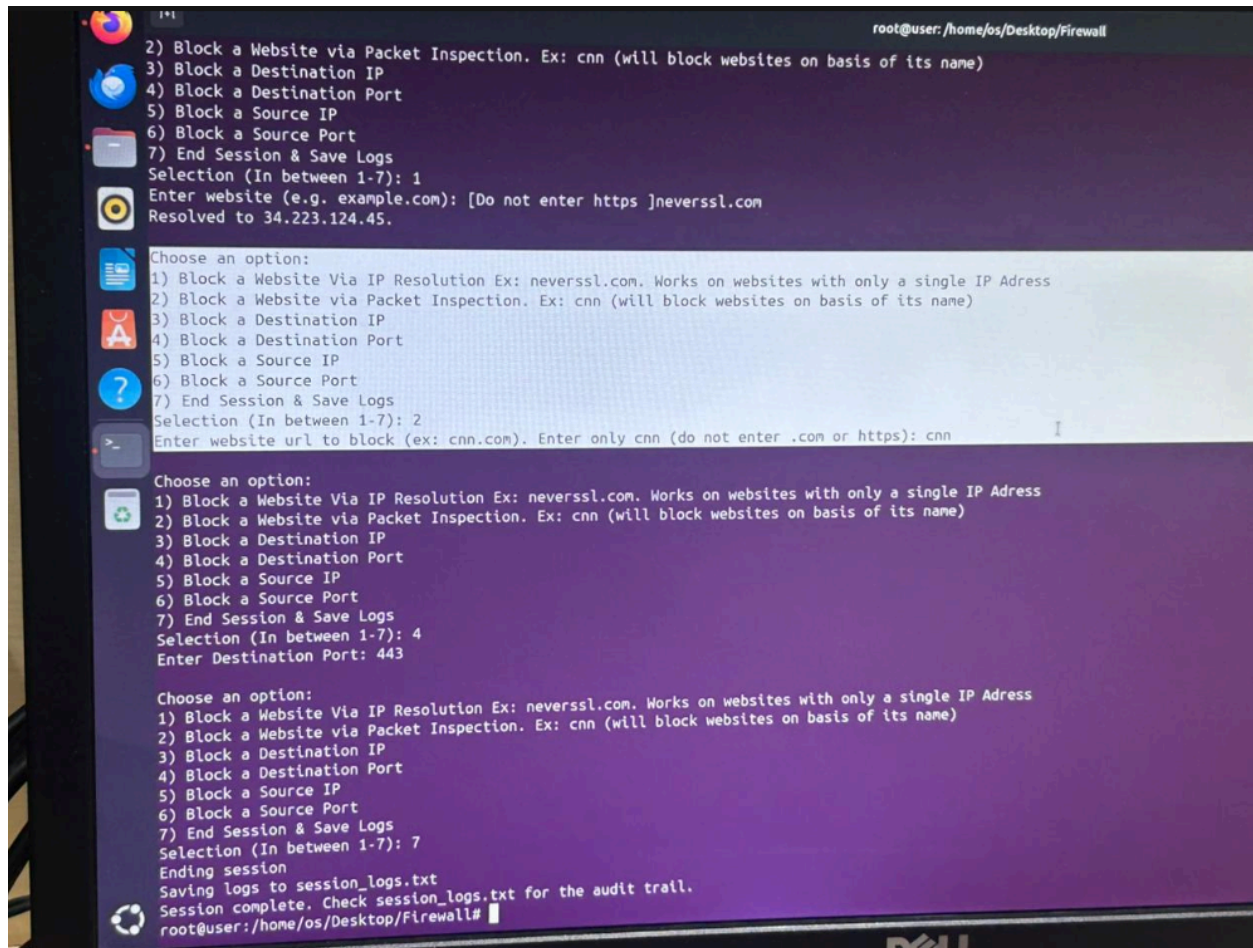


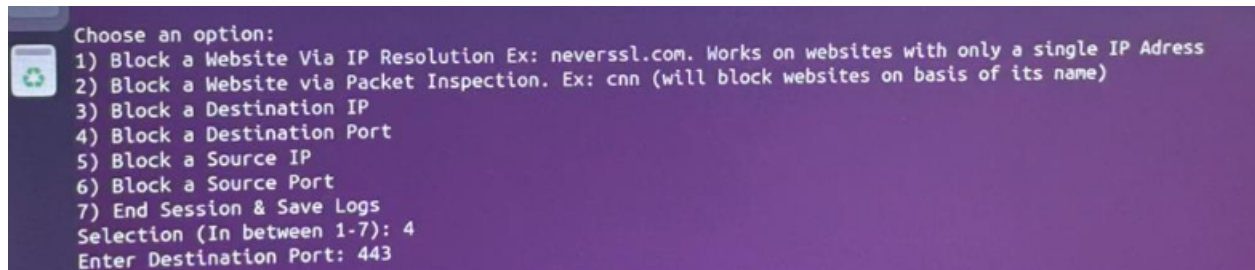
Figure 5: Adding a website restriction

Test 3: Destination IP Blocking

On entering a specific IP address, it will be blocked. Option 1 (blocking a website via IP resolution converts website link into an IP address and uses this mechanism to block a specific website

Test 4: Destination Port Blocking

Using Option 4, outbound traffic to Port 443 (HTTPS) was successfully added to the kernel's block list. The wrapper script handles this by translating the user input into the `dest_port 443` command.



```
Choose an option:
1) Block a Website Via IP Resolution Ex: neverssl.com. Works on websites with only a single IP Address
2) Block a Website via Packet Inspection. Ex: cnn (will block websites on basis of its name)
3) Block a Destination IP
4) Block a Destination Port
5) Block a Source IP
6) Block a Source Port
7) End Session & Save Logs
Selection (In between 1-7): 4
Enter Destination Port: 443
```

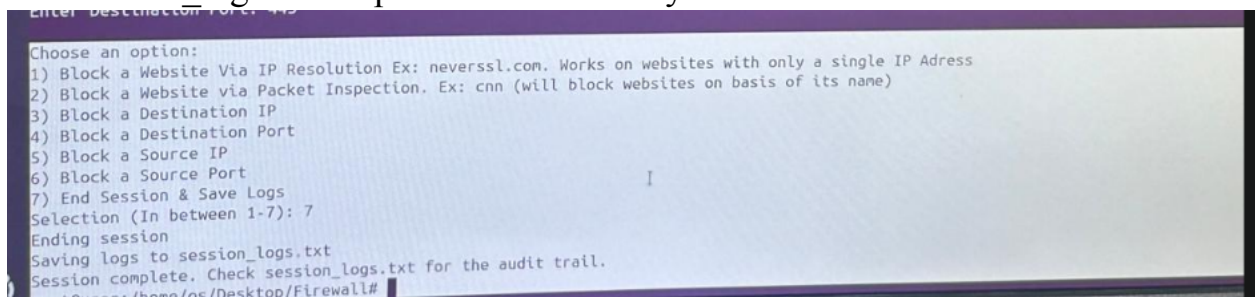
Figure 6: Block a Destination Port

Test 5 and Test 6: Source IP and Source Port Blocking

These options will allow information from selected IP addresses and ports to not be received by the system. An example of this is using port 22, which would block all inbound SSH packets.

Test 7: Session Termination and Logging

Upon selecting Option 7, the script halts the operation by unloading the module via `rmmod firewall`. It then successfully exports the kernel's audit logs, capturing all the printk statements generated during the session, and saves them into a local text file named `session_logs.txt` for post-execution analysis.



```
Choose an option:
1) Block a Website Via IP Resolution Ex: neverssl.com. Works on websites with only a single IP Address
2) Block a Website via Packet Inspection. Ex: cnn (will block websites on basis of its name)
3) Block a Destination IP
4) Block a Destination Port
5) Block a Source IP
6) Block a Source Port
7) End Session & Save Logs
Selection (In between 1-7): 7
Ending session
Saving logs to session_logs.txt
Session complete. Check session_logs.txt for the audit trail.
root@queer:/home/os/Desktop/Firewall#
```

Figure 7: Ending the interactive session and saving the kernel ring buffer audit logs.

In the continuous execution logs, the system seamlessly handles the sequential addition of multiple rules (IP, URL, and Port) in a single session without requiring a module restart. This proves the effectiveness and stability of the concurrent spinlock (`fw_lock`) design utilized within the kernel space to safely update the configuration arrays while packets are actively traversing the network stack.

5. Conclusion

5.1 Conclusion

The implementation of the custom kernel-level firewall successfully demonstrates the ability to extend the Linux kernel's networking stack for high-performance security enforcement. By integrating filtering rules directly into kernel space via the Netfilter framework, the project achieves protocol-level granularity with near-zero latency overhead.

Through the development of this loadable kernel module, the project bridged theoretical operating system concepts with practical systems development. It highlighted the critical importance of secure boundary crossings between user space and kernel space via the `/proc` filesystem, the necessity of safe concurrency practices using spinlocks (`spin_lock_bh`), and direct packet manipulation.

The user-space testing phase definitively validated the module's capability to intercept transport-layer attributes (IP addresses and Ports) and execute targeted Deep Packet Inspection on application-layer payloads (DNS requests via UDP port 53) to drop restricted traffic instantaneously.

5.2 Limitations

While highly effective, the current implementation operates with certain constraints:

- **Volatility:** The firewall rules are stored in volatile kernel memory (kmalloc arrays). Consequently, these rules do not persist across system reboots or when the module is unloaded, requiring the user to re-apply the configuration for each session.

- **Capacity:** The data structures utilize fixed-size arrays, hardcoding the firewall to support a maximum of 100 blocks per category (100 source IPs, 100 destination IPs, etc.).

- **URL Blocking Scope:** The deep packet inspection feature specifically intercepts unencrypted UDP traffic on Port 53. It searches the payload for matching strings and drops the DNS request; however, it cannot inspect DoH (DNS over HTTPS) traffic, requiring specific browser configurations to ensure testing accuracy.